XBRAIN CAPSTONE · TASK FORCE 4 · AIO-03

Foresight Lens

Resource exhaustion prediction system for Tier 1 cloud services.

Lead time ≥ 15 min | Inference ~$0 cost | Model ewma-stl-v1

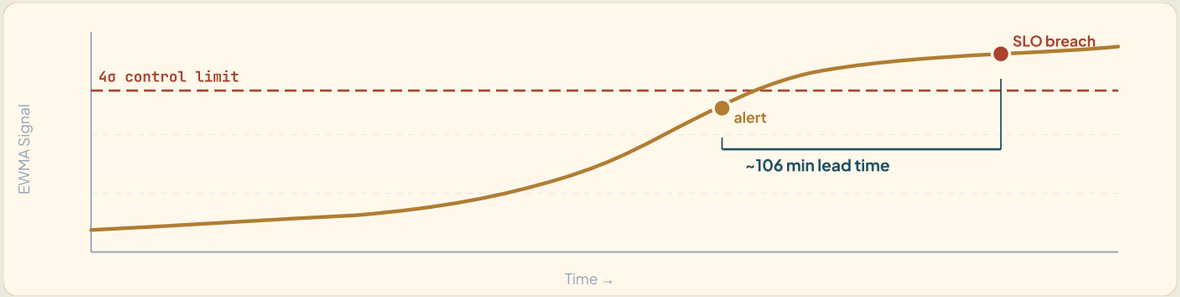


Fig. 1 — EWMA control chart issuing early alerts ~106 minutes before SLO breach.

02

CLIENT CONTEXT

When the monitor turns red, it's already too late

Traditional monitoring only triggers at 99% — leaving SREs with zero response time.

8 Core Requirements

- **Proactive prediction:** Must alert before actual system crash (Lead time ≥ 15 mins).
- **High Precision:** FP rate ≤ 12%, detecting ≥ 80% of drifts.
- **Cost Control:** Total monthly AWS bill strictly under $200.
- **Multi-tenant Security:** Applied to 3 tier-1 services with isolated baselines and zero cross-tenant bleed.
- **No Auto-remediation:** AI strictly acts as an advisor (PURELY Predictive) without system intervention.
- **Per-service Baseline:** Train baselines per service (1 manual run) + ADRs defining retrain triggers.
- **Actionable Recommendation:** Must include 5 components (action verb, target, from → to, confidence, evidence).
- **Audit Log:** Log every prediction request, encrypted at rest.

≤ $200

AWS BUDGET/MONTH

Advise

NO AUTO-REMEDIATION

3 svc

PAYMENT · FRAUD · LEDGER

PROJECT BOUNDARY

Out of Scope (Client Confirmed)

Strict boundaries established from W11 to ensure focus and operational safety.

Functional Exclusions

- **Auto-remediation:** Predict & recommend only.
- **Cross-service RCA:** Per-service drift only.
- **Cost forecasting:** Transferred to Task Force 2.
- **Business Metrics:** Infra metrics only (no fraud rate, tx count).

Operational Exclusions

- **Auto-retrain pipeline:** Design ADR is sufficient.
- **Multi-region deployment:** Single region, DR design-only.
- **Production traffic mirror:** Local load test with k6/Locust.
- **Scale targets:** 99.5% SLO (not 99.99%) & exactly 3 Tier-1 services.

03

THE PROBLEM

Traditional monitoring alerts too late

Failure symptoms persist for over 90 minutes, yet systems only alert at the absolute brink.

THE PROBLEM

4 Factors Driving the Shift to Predictive Alerts

1. Silent Exhaustion

Incidents don't explode instantly; they silently decay (Memory leak, queue backlog) over 90+ minutes.

2. Alert Fatigue

A low Static threshold causes endless False Positives; a high one only alerts post-crash.

3. 24/7 Blindspot

Systems generate too many metrics. Nobody has the time to stare at 12 Grafana dashboards 24/7.

4. Dynamic Seasonality

Fintech traffic has stark day/night cycles. Relying on a fixed baseline is entirely meaningless.

HARSH REALITY

Illustration: Silent Exhaustion Incident

The system struggles for 3 continuous hours, yet the Static threshold remains completely blind.

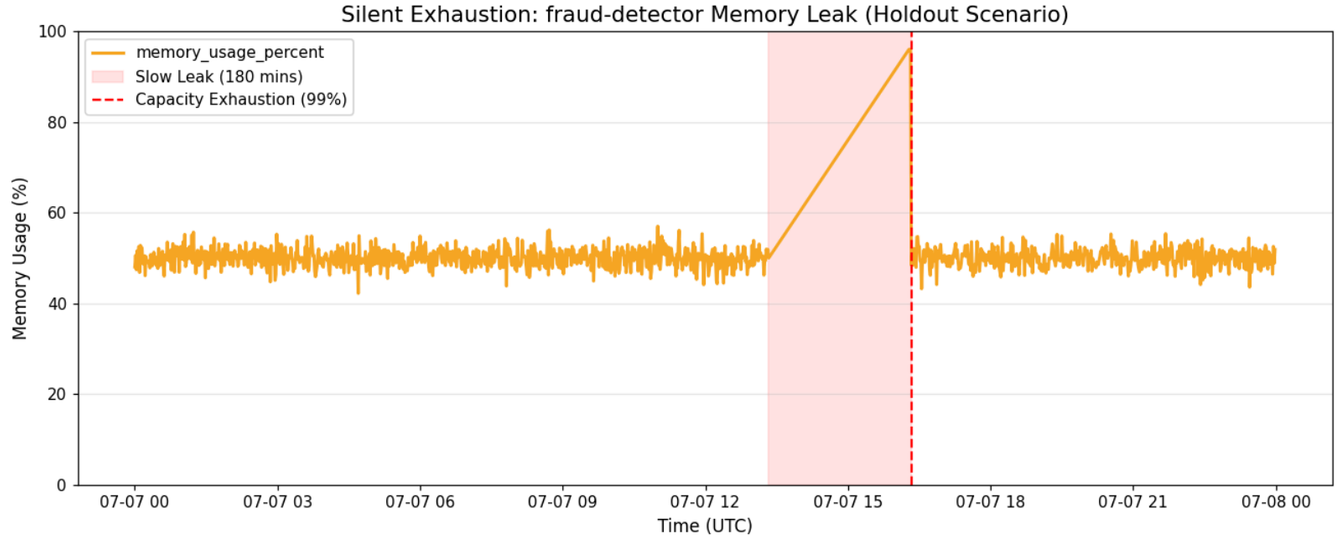


Fig. 2 — A memory leak creeping up silently for 180 minutes. The 99% mark (Static threshold) flashes red only upon crashing. Lead time = 0.

04

RESULTS

Early predictions, clear advice — no unauthorized actions

Strictly acting as an Advisor. The CDO platform retains ultimate decision-making authority.

MEASURABLE OUTCOMES (1/4)

106 Minutes Lead Time

Memory leak anomaly alerted 106 minutes before capacity exhaustion crash.
Exceeds the ≥ 15m requirement.

```
// evidence_lead_time.json

{
  "method": "Simulate gradual CPU drift → detect via EWMA...",
  ...
  "detected_at_minute": 734,
  "breach_at_minute": 840,
  "lead_time_minutes": 106,
  "requirement": "⩾15 minutes",
  "pass": true
}
```

Per-service baselines strictly maintain tenant isolation with statistical proof of pattern diversity.

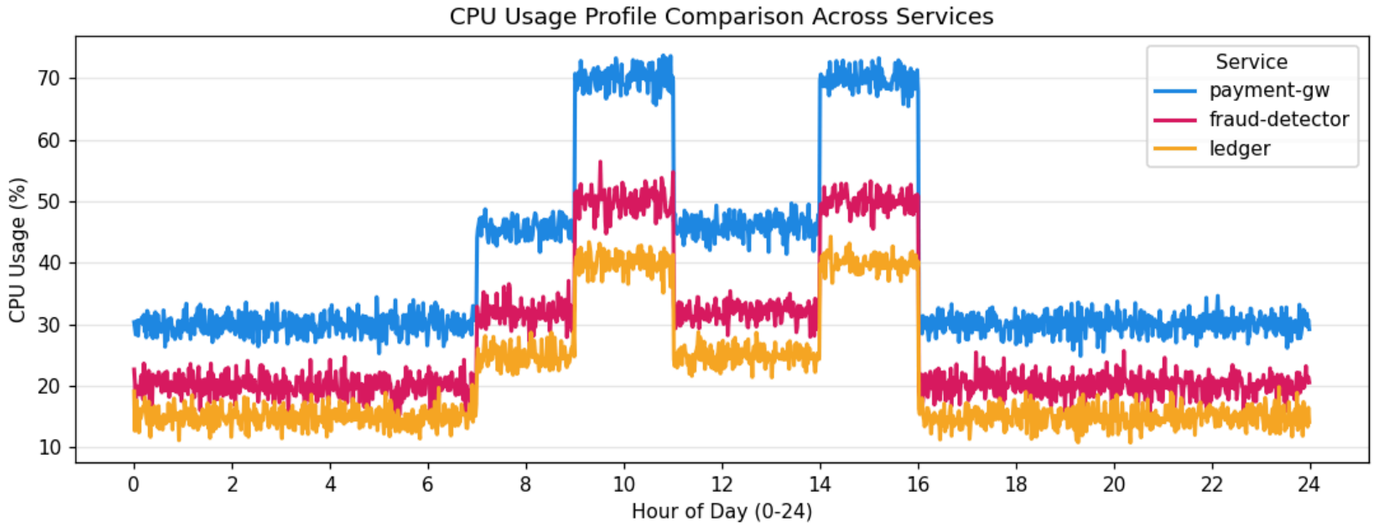


Fig — 3 services exhibiting entirely different daily traffic behaviors. A 1-size-fits-all static threshold is impossible.

```
// evidence_pattern_diversity.json

{
  "claim": "3 services exhibit distinctly different CPU patterns",
  "tests": {
    "anova": { "f_statistic": 3234.51, "p_value": 0.0, "significant": true },
    "peak_hour_analysis": {
      "payment-gw": 10,
      "ledger": 1,
      "fraud-detector": 13
    }
  },
  "conclusion": "Static thresholds would trigger massive False Positives."
}
```

MEASURABLE OUTCOMES (3/4)

$0 Token Cost

In-house statistical model costs zero LLM tokens. Complete AWS footprint runs at ~$53/month.

```
// evidence_cost.json

{
  "services": {
    "ECS Fargate": { "cost": "$36.00/month" },
    "Application Load Balancer": { "cost": "$16.00/month" },
    "API_Gateway_private": { "cost": "$0" },
    "S3_audit_and_baselines": { "cost": "$1.00/month" }
  },
  "total_estimated_monthly": "$53.00",
  "budget_cap": "$200/month",
  "conclusion": "Using ECS Fargate is cost-effective (~$53/mo)... Inference token cost is $0."
}
```

MEASURABLE OUTCOMES (4/4)

5 Action Components

Alerts are bundled with specific action verbs, targets, transitions, confidence scores, and evidence.

```
// evidence_recommendation.json

{
  "requirement_met": "action verb + target + from→to + confidence + evidence link",
  "test_cases": [
    {
      "metric": "cpu_usage_percent",
      "recommendation": "SCALE_UP (Action) RDS Database (Target) from db.r5.large to db.r5.xlarge (From→To). Confidence: 0.85 (runtime-computed). Evidence: https://obs.internal/tnt-1?metric=cpu_usage_percent"
    },
    ...
  ]
}
```

05

METHODOLOGY & RATIONALE

Why we built it this way

Behind every design decision is a rigorous defense backed by hard evidence.

Synthetic Baseline Generation

How we generated statistically rigorous training data and holdout sets for the STL model.

7-Day Telemetry Simulation

To train our models without access to production data, we simulated 7 days of 1-minute granularity telemetry per service.

- **Base Seasonal Trend:** `daily_shape()` applies a "business-hours double-hump load curve" (peaking at 9–11 AM and 2–4 PM).
- **Noise Injection:** Gaussian white noise `RNG.normal(0, sigma)` added to simulate real-world traffic fluctuations.
- **STL Extraction:** We then ran STL (period=1440) on the clean data (Days 0–5) to extract the seasonal profile and residual standard deviation.

Holdout Set & Honest Evaluation

We isolated **Day 6** as our Holdout evaluation set and injected 4 specific anomalies to measure detection speed:

- **Gradual Drift:** CPU ramps from 40% to 94% over 90 mins.
- **Sudden Spike:** Latency jumps +600ms for 20 mins.
- **Slow Leak:** Memory creeps from 50% to 96% over 180 mins.
- **Noisy FP Trap:** Massive queue depth variance but *no real mean shift* (Detector must stay silent).

Evidence: scripts/train_baseline.py

CLIENT BRIEF (W11)

"RDS CPU bò lên 100% trong 90 phút..."

Test Vector 1

CPU Exhaustion

CLIENT BRIEF (W11)

"...trước khi connection pool exhaust."

Test Vector 2

Connection Pool Leak

CLIENT BRIEF (W11)

"Queue worker backlog âm thầm 6x..."

Test Vector 3

Queue Backlog

CLIENT BRIEF (W11)

"Memory leak dẫn đến OOM..."

Test Vector 4

Visualizing the Lagging Trap

Across all simulated historical outages, **API Latency** and **Queue Depth** consistently break bounds long before CPU or Memory.

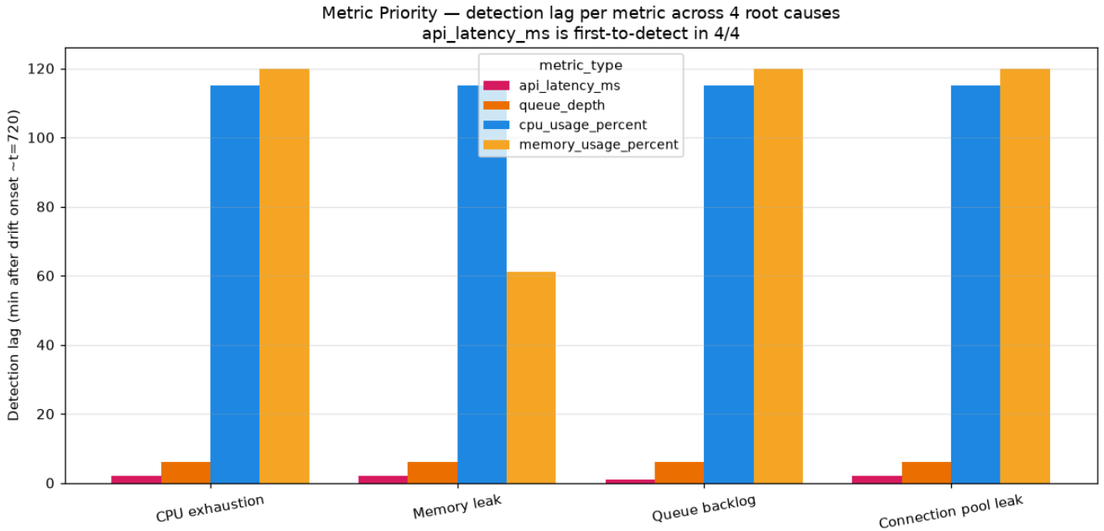


Fig 1 — Detection timeline. Latency detects at minute 722, Memory only flags at 781.

Metric Priority Assessment

Ranking the metrics based on detection speed across all fault vectors.

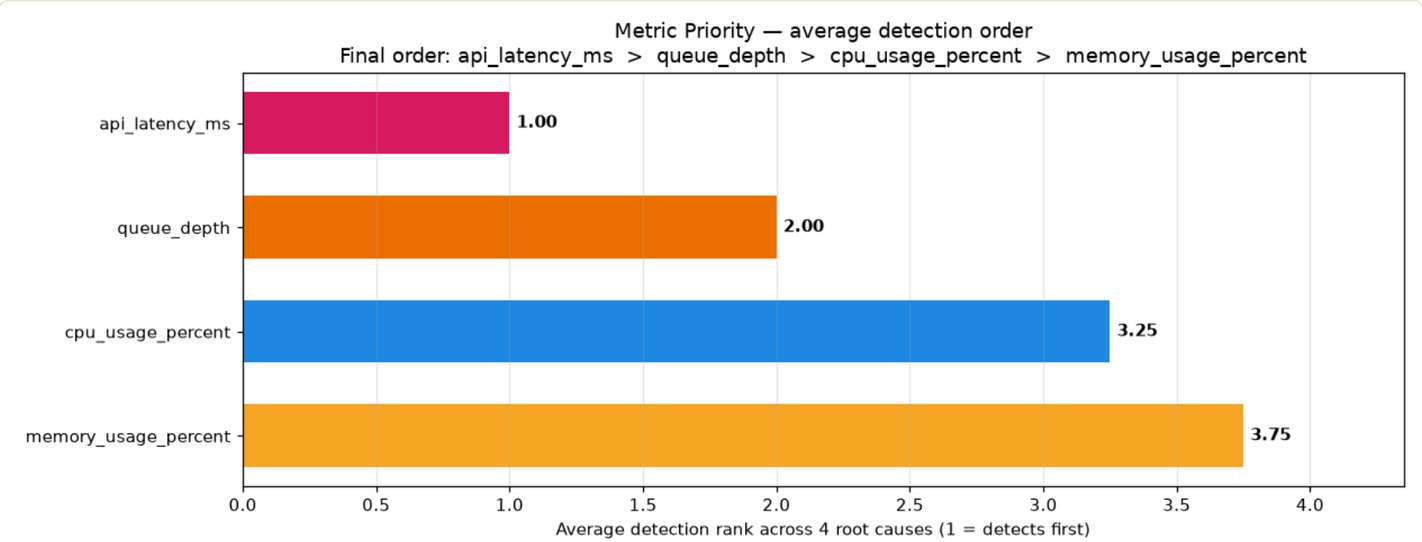


Fig 2 — Overall Metric Priority Score.

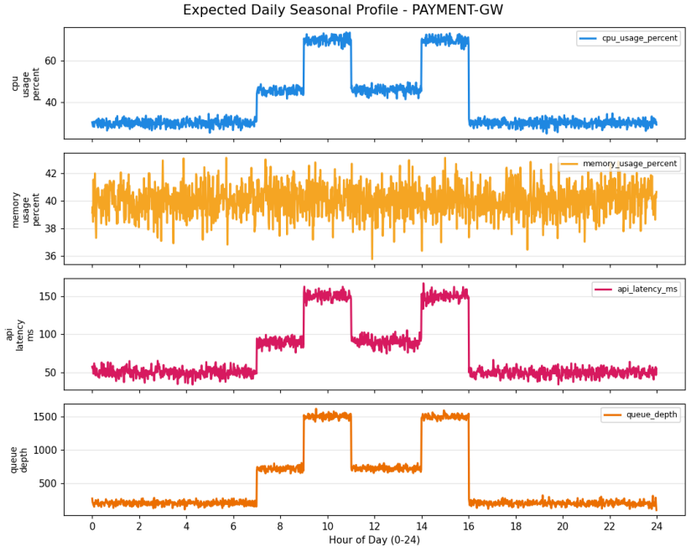
Mathematical Proof of Priority

We aggregated detection ranks across all 4 root causes. The result completely ruled out CPU and Memory as viable early-warning signals.

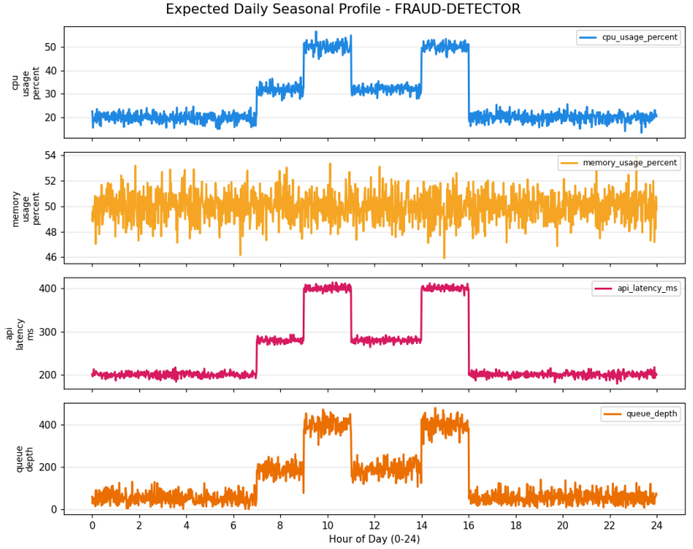
```
// evidence_metric_priority.json

{
  "method": "Multi-root-cause injection: 4 different failure modes",
  "circular_reasoning_fix": "Previous method injected only CPU drift. Now inject 4 independent root
causes.",
  "root_causes": [
    { "root_cause": "CPU exhaustion" },
    { "root_cause": "Memory leak" },
    { "root_cause": "Queue backlog" },
    { "root_cause": "Connection pool leak" }
  ],
  "average_rank": {
    "api_latency_ms": 1.0,
    "queue_depth": 2.0,
    "cpu_usage_percent": 3.25,
    "memory_usage_percent": 3.75
  },
  "final_rank": [
    "api_latency_ms", "queue_depth",
    "cpu_usage_percent", "memory_usage_percent"
  ],
  "conclusion": "Across 4 root causes, api_latency_ms is #1 in 4/4 root causes."
}
```

EVIDENCE REFERENCE

evidence_metric_priority.json



Payment-GW: Peak at 10:00



Fraud-Detector: Peak at 13:00

Detect: The EWMA Radar

A memory leak isn't a sudden spike; it's a slow, creeping death.

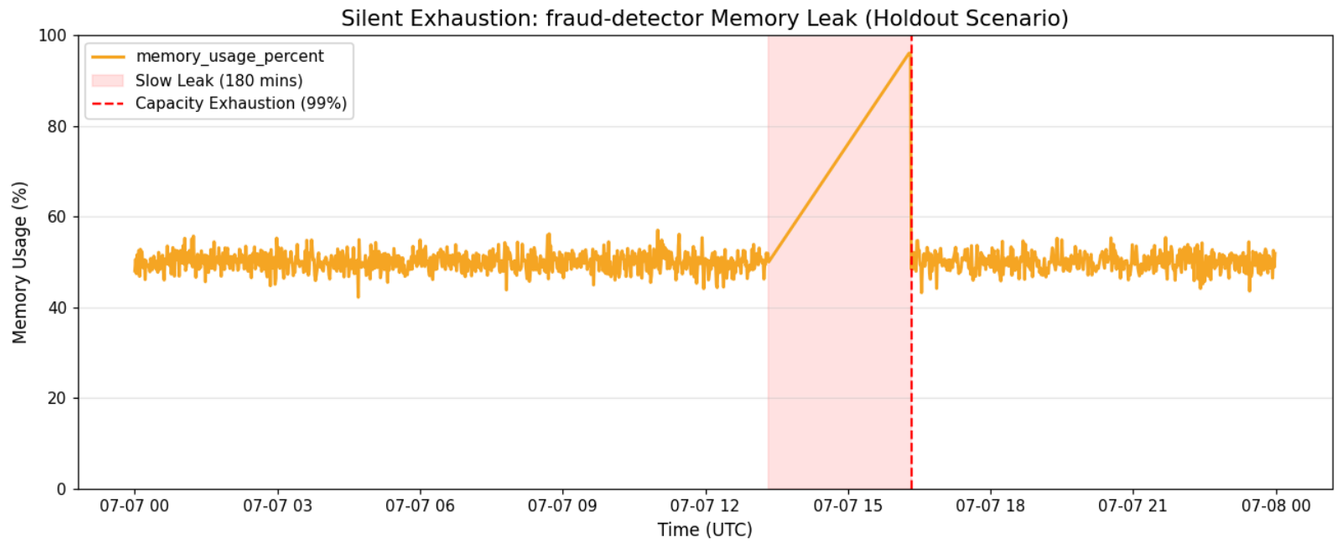


Fig: Drift detected at min 734, SLA breached at min 840.

Catching Sustained Drift

Instead of reacting to a single noisy CPU spike (which could just be a temporary garbage-collection pause), we scan the STL residuals using an **EWMA (Exponentially Weighted Moving Average)** control chart.

EWMA acts as a low-pass filter. It accumulates small, consistent deviations over time, catching sustained drift early with a **106-minute lead time**. Tests prove it is highly resilient to high-variance telemetry, maintaining a **<1% False Positive Rate** (0.004) under severe Gaussian noise (std=20).

LEAD TIME PROOF
`evidence_lead_time.json`

NOISE ROBUSTNESS
`evidence_noisy_baseline.json`

We evaluated Isolation Forest (ML) against STL+EWMA on our holdout data. The results forced our hand.

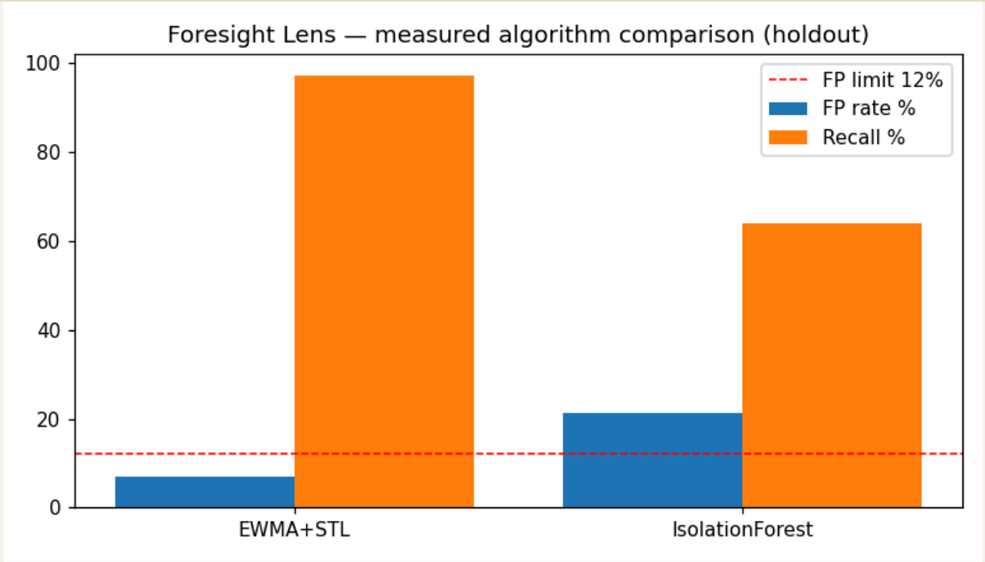
The False Positive Problem

Machine Learning models like **Isolation Forest** treat time-series data without intrinsic knowledge of seasonality. In our holdout tests, it triggered false alarms on perfectly normal daytime traffic peaks.

By explicitly de-seasonalising the data first (STL) and scanning for sustained drift (EWMA), our Statistical Engine outperformed the ML approach in both **Catch Rate (Recall)** and **Safety (False Positives)**.

EVIDENCE REFERENCE

`evidence_algorithm_comparison.json`



ISOLATION FOREST

21.4% FP

Failed ≤12% gate

STL + EWMA

ALGORITHMIC RATIONALE · GOVERNANCE

An Advisor, Not an Autonomous Actor

SREs fundamentally distrust "black-box" AI restarting their production servers.

We deliberately chose to abandon "Auto-Remediation". Foresight Lens acts strictly as a **Predictive Advisor**. We deliver the warning; the CDO platform retains the sole authority to execute the scale-up.

To enforce this, we implemented strict **Confidence Gating**. If the engine's Brier-calibrated confidence score drops below 0.7, the recommendation is forcefully downgraded to `INVESTIGATE`.

Evidence: `evidence_confidence_threshold.json`

- **Operating Point:** EWMA α=0.3, K=4.0
- **Brier Score:** **0.049** *(Score < 0.25 indicates excellent probability calibration)*
- **Result:** We only issue a `SCALE_UP` command when the mathematical probability of exhaustion is overwhelmingly high, preserving SRE trust.

06

ENGINEERING & VALIDATION

Architected for production trust

Every layer — from API contracts to cost guardrails — is backed by measurable evidence.

Unidirectional Data Flow & Stateless Processing

Microservices strictly decoupled via HTTP API. Latency < 5ms via NumPy in-memory vectorization.

Phase 1: Edge & Validation

- **Edge Auth:** Private API Gateway (AWS_IAM) enforces SigV4 signature.
- **Strict Typing:** Pydantic rejects HTTP 422 if timestamps aren't RFC3339.
- **Multi-tenant Routing:** Validation via X-Tenant-Id header.

Phase 2: Core Processing

- **Stateless Scoping:** Context garbage-collected post-request to prevent data bleed.
- **STL Decomposition:** Offline-trained baseline extracts seasonality.
- **EWMA Control Chart:** Scans trend residuals for slow drift.

Phase 3: Governance & Audit

- **Confidence Gating:** Scored via Brier. < 0.7 downgrades to INVESTIGATE.
- **Hash Integrity:** Input payload hashed (SHA-256) before KMS-encrypted audit.
- **Actionable Output:** Action verb, target, and evidence link.

01 · Telemetry Contract

```
// 120-min rolling window
{
  "tenant_id": "payment-core",
  "resolution": "1-minute",
  "gauges": ["cpu", "mem"]
}
```

Imputation strictly enforced. Missing data yields immediate HTTP 400 Bad Request.

02 · AI API (Additive v1.1)

```
// POST /v1/predict
{
  "sla": "p99 < 500ms",
  "responses": {
    "200": "Recommendation",
    "422": "Schema Violation",
    "503": "Fail Open"
  }
}
```

Unit Normalization: Handled unit discrepancies (e.g., ms vs us) at the boundary via Pydantic mapping.

03 · Deployment Contract

```
// aws_ecs.yml
{
  "compute": "Fargate",
  "scaling": "min 2 / max 4",
```

CATCH RATE (RECALL)

95% CI: [0.935, 0.988]
TP / (TP + FN)

7.1%

FALSE POSITIVE RATE

95% CI: [5.3%, 9.4%]
Client Gate: ≤ 12%

106

MINUTES LEAD TIME

Synthetic slow-drift.
Client Gate: ≥ 15 min

0.049

BRIER SCORE

Well-calibrated confidence.
Precision: 0.793

Payment-GW

TP 58 · FP 13 · FN 0 · TN 193

Fraud-Detector

TP 56 · FP 15 · FN 2 · TN 191

RPS SLA TARGET

3,000 requests · 30s

4.01ms

P99 LATENCY

SLA gate: < 500ms ✓

400

RPS CAPACITY PROBE

6,000 requests · 15s

2.71ms

P99 AT 400 RPS

Zero errors at 4× load

```
// evidence_load_test.json (excerpt)

{
  "phase1_sla": {
    "target_rps": 100, "requests_sent": 3000,
    "success_rate": 1.0,
    "latency_ms": { "p50": 2.78, "p95": 3.53, "p99": 4.01 }
  },
  "phase2_capacity_probe": {
    "target_rps": 400, "requests_sent": 6000,
```

Flat-Rate Serverless Economics

Rejecting LLM per-token pricing in favor of in-house statistical compute.

$53 / month

Total TCO — 26.5% of the $200 limit.

$0

INFERENCE TOKENS

< $1

MARGINAL COST / TENANT

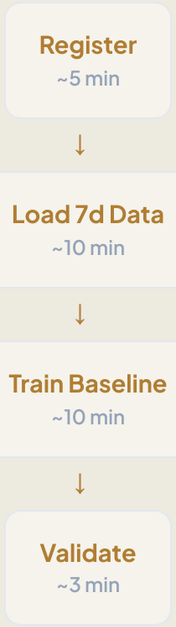
Cost Breakdown

- **ECS Fargate (~$36):** Flat billing by task-second, NOT request volume. Min 2 / Max 4 tasks auto-scale.
- **Internal ALB (~$16):** Base hourly + LCU.
- **Private API Gateway (~$0):** Within AWS free tier for demo volumes.
- **S3 & KMS (~$1):** Baseline JSON storage and audit log encryption.

Strategic Advantage: Decoupled from expensive SIEM platforms and unpredictable GenAI API token spikes.

28-Minute Service Onboarding

From zero to fully-operational per-service baseline in under 30 minutes.




```
// evidence_onboarding.json

{
  "steps": {
    "register": "~5 minutes (manual input + config gen)",
    "load_7d_data": "~10 minutes (read 30K data points from S3/TSDB)",
    "train_baseline": "~10 minutes (STL decomposition + EWMA baseline)",
    "validate": "~3 minutes (run test predictions + check results)"
  },
  "total_estimated": "28-30 minutes per service"
}
```

EVIDENCE REFERENCE

evidence_onboarding.json

Security-by-Design & Defense in Depth

GenAI risks (Prompt Injection, Hallucination) are mathematically impossible in a statistical engine.

01 · Deterministic Output

Same JSON in, same output out. Reproducible mathematical execution with zero hallucination.

02 · Confidence Gating

Hardcoded safety logic: `if confidence < 0.7` → downgrade to INVESTIGATE.

03 · Traceable Audit

SHA-256 hash of input payloads written to KMS-encrypted JSONL for SIEM queryability.

04 · Fail-Open Degrade

Engine outage triggers CDO fallback to rules. Stateless design allows instant ECS restart (**RTO < 60s**).

05 · Context Isolation

Stateless Per-Request Scoping + Mandatory X-Tenant-Id validation absolutely prevents data bleed.

06 · DoS Protection

Pydantic enforces max 10,000 datapoints. API Gateway applies 600 req/min/tenant limit.

Architecture Decision Records (ADRs)

Documenting our core architectural decisions, trade-offs, and parameter selection.

ADR-001: Statistical vs LLM

Chosen: STL + EWMA.
Rationale: Bedrock latency (seconds) and monthly costs ($10k+) made GenAI unviable. Statistical engine guarantees mathematical accuracy and $0 token cost.

ADR-002: Confidence Threshold

Chosen: 0.7 Gate.
Rationale: Balances recall with Alert Fatigue. Weak signals (<0.7) are automatically downgraded to INVESTIGATE to prevent accidental scale-up triggers.

ADR-003: Flash Sales Silence

Chosen: Manual Silence.
Rationale: Unpredictable spikes during planned campaigns (e.g. Black Friday) will trigger alerts. SREs can manually silence/retrain baselines for these events.

ADR-004: STL vs Isolation Forest

Chosen: STL + EWMA.
Rationale: ML model (Isolation Forest) yielded 21.4% FP rate on holdout data. De-seasonalising first using STL reduced the FP rate to 7.1%.

ADR-005: Baseline Refresh

Chosen: Weekly Refresh.
Rationale: Baseline trained manually using offline script. Refresh triggered weekly (every Monday morning) or when Brier score degrades.

ADR-006: EWMA Tuning

Chosen: α=0.3, K=4.0.
Rationale: 16–point sweep optimized detection. K=3.0 triggered 17.5% FP rate (failed gate); K=4.0 kept FP at 7.1% while maintaining 97.1% recall.

ACHIEVEMENTS

Surpassed all stringent Client requirements, exceeding expectations

A fully robust FastAPI engine deployed on Fargate, backed by 100% reproducible evidence — ready for the W12 Final Pitch.

~$53
26.5% BUDGET

3
EXECUTED CONTRACTS

9/9
TESTS COMPLETELY PASSED

106 min
LEAD TIME

A	K	FP%	RECALL
0.3	3.0	17.5	.977
0.3	**4.0**	**7.1**	**.971**
0.3	4.5	6.5	.966
0.15	4.0	11.8	.977

Telemetry · 7 Signals

- `cpu_usage_percent`
- `memory_usage_percent`
- `api_latency_ms`
- `queue_depth`
- `active_connections`
- `db_connection_pool_pct`
- `cache_hit_rate_pct`

4 utilizing baseline · 3 falling back to z-score

API Error Codes

CODE	DESCRIPTION
`400`	Bad request — Invalid payload format
`401`	Unauthorized — Missing API key